

P0110R0: Implementing the strong guarantee for `variant<>` assignment

N4542 has drawn a lot of comments across the web, from me and others. The fundamental problem from my perspective is the undefined behaviour that arises from the *invalid* state. My initial reaction was to propose an *empty* state in its place, the key distinction being that (a) the user could explicitly set the variant to be *empty*, and (b) operations on *empty* variants either succeeded without problem (e.g. copying an *empty* variant just created a second *empty* variant), or threw an exception (e.g. calling `get` on an empty variant), rather than having undefined behaviour.

However, some people were strongly against the idea of an empty state, since they want to be able to rely on their variants holding exactly one of the types in the list, and an empty state is in many ways equivalent to an additional type in the list. Rather than handling `variant<int,string>`, code must now handle `variant<int,string,empty>`. Several people proposed that we require `variant<>` to implement the *strong guarantee* for assignment, so either the assignment is successful, or the value is unchanged if an exception was thrown. I then set out to see how efficiently we could manage this.

The strong guarantee for assignment

The difficulty with providing the strong guarantee is that we want `variant` to hold any kind of type, so operations might throw exceptions, or not be available at all. We therefore need to identify the possible scenarios and ensure that the everything works OK.

Assumption: if the contained type is the same as the type being assigned, then we use the assignment operator of that type, just as in N4542.

For all the following, I'm going to assume we have a variant `v` of type `variant<A,B,C,D>` that holds an instance of type `A`, and we are doing an operation that will construct an instance of type `B`. In each scenario I'll describe the properties of `A`, `B`, `C` and `D` that I'm assuming for that scenario.

Case 1: The easy case

If we're assigning to `v` using an operation that creates a type `B` using a `noexcept` constructor, then it's all straightforward:

1. Destroy the contained `A` object (which can't throw as we require `noexcept` destructors).
2. Construct the new `B` object (which can't throw as the constructor we're using is `noexcept`).
3. Mark the variant as holding a `B` (which can't throw).

This is what we get if our type `B` is a built-in type, or we're move-constructing a type with a `noexcept` move constructor like `std::string`, e.g.

```
variant<int,std::string> v(42);  
v=std::string("hello");
```

If we're assigning to `v` using an operation that creates a type `B`, but `B` has no `noexcept` constructors (e.g. any C++98 legacy type), then step 2 above may throw. This would leave `v` in an inconsistent state, so we need to do something to avoid it.

Case 2: `B` is *nothrow-move-constructible*

If `std::nothrow_move_constructible<B>::value` is true, then things are almost as easy. Rather than constructing our new `B` directly in the variant, we can first construct an instance on the stack. If this throws then there isn't a problem, but if it succeeds then we can move-construct it into the variant storage without any exceptions, and we're home and dry:

1. Construct a new `B` on the stack. If this throws then `v` is unchanged.
2. Destroy the contained `A` object (which can't throw as we require `noexcept` destructors).
3. Move-construct a `B` object in the variant from our local `B` object (which can't throw as the move constructor is `noexcept`).
4. Mark the variant as holding a `B` (which can't throw).
5. Destroy the local `B` object on return from the function (which can't throw as we require `noexcept` destructors).

This safely provides us with the strong guarantee at the cost of an extra move operation, but what if `B` isn't *nothrow-move-constructible*?

Case 3: `B` isn't *nothrow-move-constructible*, but `A`, `C` and `D` are

If all the other types in the variant are *nothrow-move-constructible*, then it doesn't matter whether or not `B` is, because we can safely save the old value, and then restore it if our `B` construction fails:

1. Move-construct the existing `A` onto the stack (which can't throw as `A` is *nothrow-move-constructible*).
2. Destroy the contained `A` object (which can't throw as we require `noexcept` destructors).
3. Construct the new `B` object (which may throw).
4. Either:
 - a. The constructor didn't throw, so mark the variant as holding a `B` (which can't throw), or
 - b. The constructor did throw, so move-construct the `A` from the stack back into the variant storage (which can't throw as `A` is *nothrow-move-constructible*), and rethrow the exception.
5. Destroy the local `A` object on return from the function (which can't throw as we require `noexcept` destructors).

Though the execution sequence only cares about `A` and `B`, we can only use this if `C` and `D` are also *nothrow-move-constructible*, since we cannot tell at compile time which type is contained in the variant.

This again safely provides the strong guarantee at the cost of an extra move (and a second extra move in the case of an exception), but what if we can't do that either because `A` is not *nothrow-move-constructible*?

Case 4: The extreme case: neither `A` or `B` are *nothrow-move-constructible*

If `A` is non-movable, or has a move constructor that may throw then we cannot touch the contained `A` object in case things go terribly wrong, meaning we end up with neither a `B` nor an `A`. We're trying to implement the strong guarantee, so those are the only two possibilities here.

The solution is thus to provide a second buffer to hold our `B` object, and construct it before we destroy the old value, as follows:

1. Construct the new `B` object in the buffer. If this throws then `v` is unchanged.
2. Destroy the contained `A` object (which can't throw as we require `noexcept` destructors).
3. Mark the variant as holding a `B` *in the buffer* (which can't throw).

The question is: where does the buffer live? One option is to keep it in the variant instance, thus making the variant larger, and the second is to dynamically allocate it, and store a pointer to the buffer in the variant itself.

The dynamically allocated buffer is unacceptable to many users, and the circumstances under which the secondary buffer is necessary are going to be increasingly rare, so I feel comfortable requiring that it be part of the variant. As we shall see below, this does not require doubling the size of the variant, as there are techniques that can minimize the buffer size required.

Double buffering

Case 1 is only detectable on a use-by-use basis, as types may have throwing constructors that we don't use, and there is no type trait for "all constructors for this type are `noexcept`". All decisions about buffering therefore need to be done based on the availability of `noexcept` move constructors.

Based on our four scenarios above, we only need double-buffering in the variant if there are two or more types that are not *nothrow-move-constructible*. The question then is: how big a buffer do we need?

The answer is simple: we need a backup buffer that is big enough to hold the largest of the types that is not *nothrow-move-constructible*. We only need the buffer if the type being assigned is not *nothrow-move-constructible*, so we'll only double the required storage space if the largest type may throw when moved.

Double buffering enables optimization

You may have noticed that the operation list for case 4 is the same 3 operations as case 1, just in a different order, whereas cases 2 and 3 have additional move operations. This means that double-buffering is more efficient than cases 2 and 3. Therefore we want to use it where we can, without compromising our minimal space requirements.

Thus, if our type `B` will fit in our buffer, it is thus more efficient to use the operations from case 4, even if case 2 would have applied. If double-buffering is enabled, we will never hit case 3, because there will always be at least one of the other types that is not *nothrow-move-constructible*. For large types that don't fit in the buffer, we can still use case 2 if case 1 doesn't apply.

emplace

`emplace` is subtly different. The implication of `emplace` is that it is a direct in-place construction. Doing anything else is therefore somewhat disingenuous. This therefore rules out case 2 from the `emplace` implementation, and also rules out using assignment when the type is the same as the original. Supporting the strong guarantee for `emplace` would therefore require double-buffering in more cases: if any types are not *nothrow-move-constructible* we would have to double-buffer, as we cannot rely on being able to move the source out of the way.

For my implementation I have chosen to make `emplace` provide only the basic guarantee, and revert to an "empty" state if an exception is thrown.

Sample implementation

I have an implementation of `variant` that provides the strong guarantee for assignment on bitbucket, under the BSD license. The code and tests are available [here](#).